1. Import Libraries

In [1]:
```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
import os
```

2. Dataset Paths

In [2]:
```
train_dir = "C:/Users/ABHISHEK JAYAN/Downloads/Brain Cancer/train"
val_dir = "C:/Users/ABHISHEK JAYAN/Downloads/Brain Cancer/val"
test_dir = "C:/Users/ABHISHEK JAYAN/Downloads/Brain Cancer/test"
```

3. Data Preprocessing & Augmentation

In [3]:
```
img_size = (128,128)
batch_size = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2
)

test_datagen = ImageDataGenerator(rescale=1./255)
```

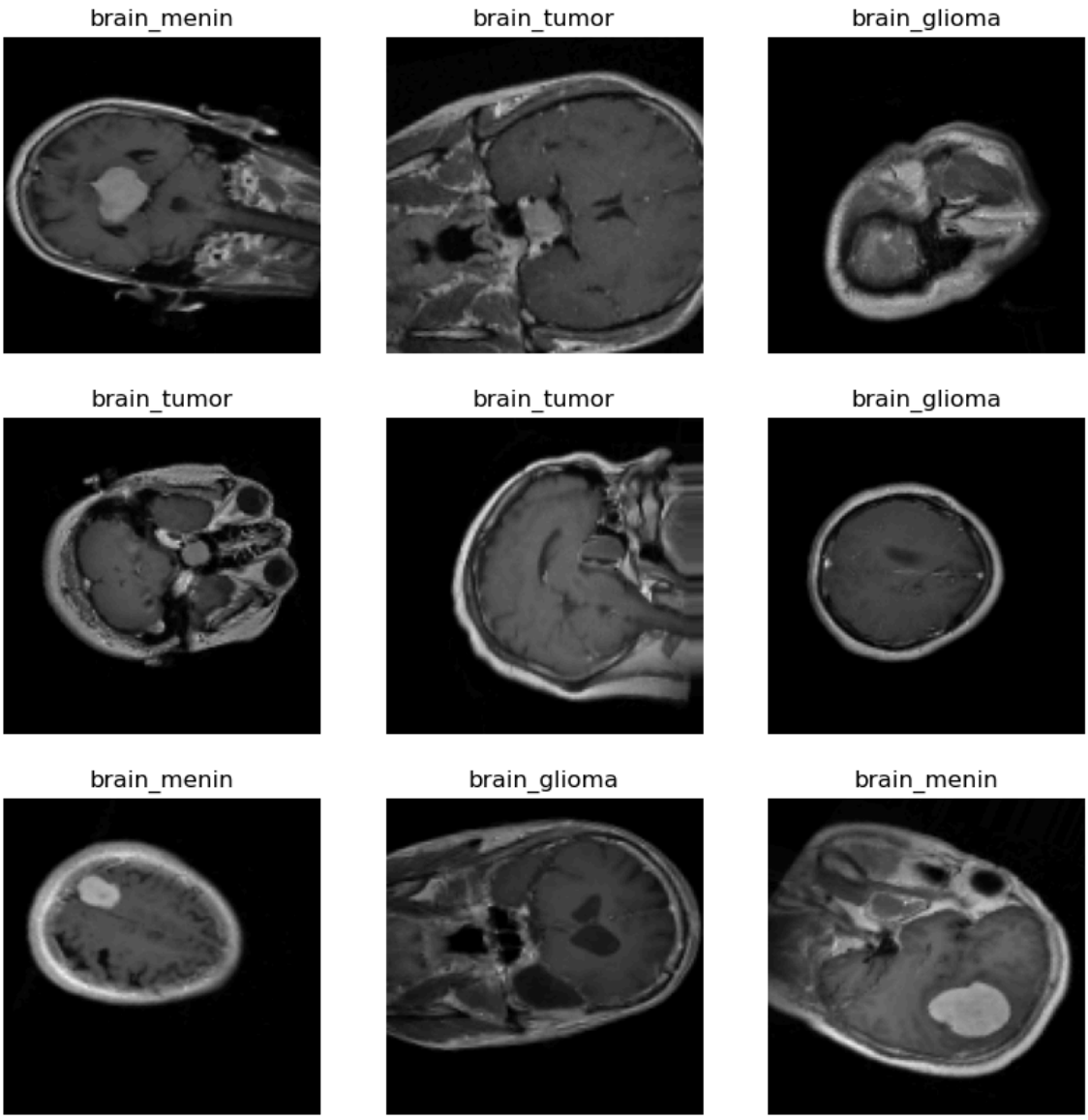
4. Load Dataset

In [4]:
```
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

validation_generator = train_datagen.flow_from_directory(
    val_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)

classnames = list(train_generator.class_indices.keys())
num_classes = len(classnames)

print("Classes:", num_classes)
```

```
Found 12000 images belonging to 3 classes.
Found 2850 images belonging to 3 classes.
Found 150 images belonging to 3 classes.
Classes: 3
```

5. Visualize Sample Images

In [5]:
```
images, labels = next(train_generator)

plt.figure(figsize=(10,10))

for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(images[i])
    plt.title(classnames[np.argmax(labels[i])])
    plt.axis("off")

plt.show()
```



6. Model 1 — Shallow Neural Network

In [6]:
```
model_shallow_nn = models.Sequential([
    layers.Flatten(input_shape=(128,128,3)),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(num_classes, activation='softmax')
])

model_shallow_nn.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model_shallow_nn.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten (Flatten)           (None, 49152)             0

 dense (Dense)               (None, 512)               25166336

 dropout (Dropout)           (None, 512)               0

 dense_1 (Dense)             (None, 3)                 1539

=================================================================
Total params: 25,167,875
Trainable params: 25,167,875
Non-trainable params: 0
_________________________________________________________________
```

7. Model 2 — Deep Feedforward Network

In [7]:
```
model_deep_nn = models.Sequential([

    layers.Flatten(input_shape=(128,128,3)),

    layers.Dense(1024, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.4),

    layers.Dense(768, activation='relu'),
    layers.BatchNormalization(),

    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.4),

    layers.Dense(256, activation='relu'),

    layers.Dense(128, activation='relu'),

    layers.Dense(64, activation='relu'),

    layers.Dense(num_classes, activation='softmax')  # 38 classes
])

model_deep_nn.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model_deep_nn.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten_1 (Flatten)         (None, 49152)             0

 dense_2 (Dense)             (None, 1024)              50332672

 batch_normalization (BatchN  (None, 1024)             4096
 ormalization)

 dropout_1 (Dropout)         (None, 1024)              0

 dense_3 (Dense)             (None, 768)               787200

 batch_normalization_1 (Batc  (None, 768)              3072
 hNormalization)

 dense_4 (Dense)             (None, 512)               393728

 batch_normalization_2 (Batc  (None, 512)              2048
 hNormalization)

 dropout_2 (Dropout)         (None, 512)               0

 dense_5 (Dense)             (None, 256)               131328

 dense_6 (Dense)             (None, 128)               32896

 dense_7 (Dense)             (None, 64)                8256

 dense_8 (Dense)             (None, 3)                 195

=================================================================
Total params: 51,695,491
Trainable params: 51,690,883
Non-trainable params: 4,608
_________________________________________________________________
```

8. Model 3 — Shallow CNN

In [8]:
```
model_shallow_cnn = models.Sequential([

    layers.Conv2D(32,(3,3),activation='relu',input_shape=(128,128,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128,activation='relu'),
    layers.Dense(num_classes,activation='softmax')
])

model_shallow_cnn.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model_shallow_cnn.summary()
```

```
Model: "sequential_2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 126, 126, 32)      896

 max_pooling2d (MaxPooling2D  (None, 63, 63, 32)       0
 )

 conv2d_1 (Conv2D)           (None, 61, 61, 64)        18496

 max_pooling2d_1 (MaxPooling  (None, 30, 30, 64)       0
 2D)

 flatten_2 (Flatten)         (None, 57600)             0

 dense_9 (Dense)             (None, 128)               7372928

 dense_10 (Dense)            (None, 3)                 387

=================================================================
Total params: 7,392,707
Trainable params: 7,392,707
Non-trainable params: 0
_________________________________________________________________
```

9. Model 4 — Deep CNN (Best Model)

In [9]:
```
from tensorflow.keras import layers, models, Input

# Number of classes
num_classes = 3

# Input layer
inputs = Input(shape=(128,128,3))

# ---------------------------
# Block 1
# ---------------------------
x = layers.Conv2D(32,(3,3),padding='same',activation='relu')(inputs)
x = layers.BatchNormalization()(x)

x = layers.Conv2D(32,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

x = layers.MaxPooling2D()(x)

# ---------------------------
# Block 2
# ---------------------------
x = layers.Conv2D(64,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

x = layers.Conv2D(64,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

x = layers.MaxPooling2D()(x)

# ---------------------------
# Block 3 (Residual Block)
# ---------------------------
skip = x

x = layers.Conv2D(128,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

x = layers.Conv2D(128,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

# Match channel dimensions
skip = layers.Conv2D(128,(1,1),padding='same')(skip)

# Skip connection
x = layers.Add()([x, skip])

x = layers.MaxPooling2D()(x)

# ---------------------------
# Block 4
# ---------------------------
x = layers.Conv2D(256,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

x = layers.Conv2D(256,(3,3),padding='same',activation='relu')(x)
x = layers.BatchNormalization()(x)

x = layers.MaxPooling2D()(x)

# ---------------------------
```

```
# Global Pooling
# ---------------------------
x = layers.GlobalAveragePooling2D()(x)

# ---------------------------
# Dense Layers
# ---------------------------
x = layers.Dense(256,activation='relu')(x)
x = layers.Dropout(0.5)(x)

# Output layer for 3 classes
outputs = layers.Dense(3,activation='softmax')(x)

# Build model
model_deep_cnn = models.Model(inputs,outputs)

# Compile
model_deep_cnn.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Show model
model_deep_cnn.summary()
```

```
Model: "model"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 input_1 (InputLayer)           [(None, 128, 128, 3  0           []
                                )]

 conv2d_2 (Conv2D)              (None, 128, 128, 32  896         ['input_1[0][0]']
                                )

 batch_normalization_3 (BatchNo  (None, 128, 128, 32  128        ['conv2d_2[0][0]']
 rmalization)                   )

 conv2d_3 (Conv2D)              (None, 128, 128, 32  9248        ['batch_normalization_3[0][0]']
                                )

 batch_normalization_4 (BatchNo  (None, 128, 128, 32  128        ['conv2d_3[0][0]']
 rmalization)                   )

 max_pooling2d_2 (MaxPooling2D)  (None, 64, 64, 32)  0           ['batch_normalization_4[0][0]']

 conv2d_4 (Conv2D)              (None, 64, 64, 64)   18496       ['max_pooling2d_2[0][0]']

 batch_normalization_5 (BatchNo  (None, 64, 64, 64)  256         ['conv2d_4[0][0]']
 rmalization)

 conv2d_5 (Conv2D)              (None, 64, 64, 64)   36928       ['batch_normalization_5[0][0]']

 batch_normalization_6 (BatchNo  (None, 64, 64, 64)  256         ['conv2d_5[0][0]']
 rmalization)

 max_pooling2d_3 (MaxPooling2D)  (None, 32, 32, 64)  0           ['batch_normalization_6[0][0]']

 conv2d_6 (Conv2D)              (None, 32, 32, 128)  73856       ['max_pooling2d_3[0][0]']

 batch_normalization_7 (BatchNo  (None, 32, 32, 128)  512        ['conv2d_6[0][0]']
 rmalization)

 conv2d_7 (Conv2D)              (None, 32, 32, 128)  147584      ['batch_normalization_7[0][0]']

 batch_normalization_8 (BatchNo  (None, 32, 32, 128)  512        ['conv2d_7[0][0]']
 rmalization)

 conv2d_8 (Conv2D)              (None, 32, 32, 128)  8320        ['max_poo
```

```
ling2d_3[0][0]']

 add (Add)                      (None, 32, 32, 128)  0           ['batch_n
ormalization_8[0][0]',
                                                                   'conv2d_
8[0][0]']

 max_pooling2d_4 (MaxPooling2D)  (None, 16, 16, 128)  0          ['add[0]
[0]']

 conv2d_9 (Conv2D)              (None, 16, 16, 256)  295168      ['max_poo
ling2d_4[0][0]']

 batch_normalization_9 (BatchNo  (None, 16, 16, 256)  1024       ['conv2d_
9[0][0]']
 rmalization)

 conv2d_10 (Conv2D)             (None, 16, 16, 256)  590080      ['batch_n
ormalization_9[0][0]']

 batch_normalization_10 (BatchN  (None, 16, 16, 256)  1024       ['conv2d_
10[0][0]']
 ormalization)

 max_pooling2d_5 (MaxPooling2D)  (None, 8, 8, 256)   0           ['batch_n
ormalization_10[0][0]']

 global_average_pooling2d (Glob  (None, 256)         0           ['max_poo
ling2d_5[0][0]']
 alAveragePooling2D)

 dense_11 (Dense)               (None, 256)          65792       ['global_
average_pooling2d[0][0]'
                                                                   ]

 dropout_3 (Dropout)            (None, 256)          0           ['dense_1
1[0][0]']

 dense_12 (Dense)               (None, 3)            771         ['dropout
_3[0][0]']

==========================================================================
========================
Total params: 1,250,979
Trainable params: 1,249,059
Non-trainable params: 1,920
__________________________________________________________________________
_______________________
```

10. Training

In [10]:
```
from tensorflow.keras.callbacks import EarlyStopping

epochs = 10

early_stop = EarlyStopping(
    monitor='val_loss',     # watch validation loss
#     patience=5,             # wait 5 epochs before stopping
    restore_best_weights=True
)
```

In [11]:
```
history_shallow_nn = model_shallow_nn.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
375/375 [==============================] - 272s 723ms/step - loss: 1.4059
- accuracy: 0.5181 - val_loss: 0.8873 - val_accuracy: 0.5312
Epoch 2/10
375/375 [==============================] - 259s 692ms/step - loss: 0.9109
- accuracy: 0.5414 - val_loss: 0.8745 - val_accuracy: 0.5726
Epoch 3/10
375/375 [==============================] - 715s 2s/step - loss: 0.8954 - a
ccuracy: 0.5474 - val_loss: 0.8535 - val_accuracy: 0.5709
Epoch 4/10
375/375 [==============================] - 247s 660ms/step - loss: 0.8919
- accuracy: 0.5501 - val_loss: 0.8725 - val_accuracy: 0.5568
```

In [12]:
```
history_deep_nn = model_deep_nn.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
375/375 [==============================] - 457s 1s/step - loss: 0.7970 - a
ccuracy: 0.6488 - val_loss: 0.7701 - val_accuracy: 0.6614
Epoch 2/10
375/375 [==============================] - 396s 1s/step - loss: 0.7467 - a
ccuracy: 0.6787 - val_loss: 0.7358 - val_accuracy: 0.6786
Epoch 3/10
375/375 [==============================] - 385s 1s/step - loss: 0.7366 - a
ccuracy: 0.6842 - val_loss: 0.7329 - val_accuracy: 0.6737
Epoch 4/10
375/375 [==============================] - 389s 1s/step - loss: 0.7213 - a
ccuracy: 0.6852 - val_loss: 0.7146 - val_accuracy: 0.6919
Epoch 5/10
375/375 [==============================] - 398s 1s/step - loss: 0.7219 - a
ccuracy: 0.6917 - val_loss: 0.7087 - val_accuracy: 0.6989
Epoch 6/10
375/375 [==============================] - 408s 1s/step - loss: 0.7139 - a
ccuracy: 0.6897 - val_loss: 0.7550 - val_accuracy: 0.6726
```

In [13]:
```
history_shallow_cnn = model_shallow_cnn.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
375/375 [==============================] - 275s 730ms/step - loss: 0.7355
- accuracy: 0.6742 - val_loss: 0.6038 - val_accuracy: 0.7512
Epoch 2/10
375/375 [==============================] - 268s 714ms/step - loss: 0.5794
- accuracy: 0.7571 - val_loss: 0.5954 - val_accuracy: 0.7565
Epoch 3/10
375/375 [==============================] - 266s 708ms/step - loss: 0.4737
- accuracy: 0.8077 - val_loss: 0.4423 - val_accuracy: 0.8260
Epoch 4/10
375/375 [==============================] - 264s 705ms/step - loss: 0.4350
- accuracy: 0.8238 - val_loss: 0.3856 - val_accuracy: 0.8442
Epoch 5/10
375/375 [==============================] - 256s 682ms/step - loss: 0.3958
- accuracy: 0.8423 - val_loss: 0.4239 - val_accuracy: 0.8365
```

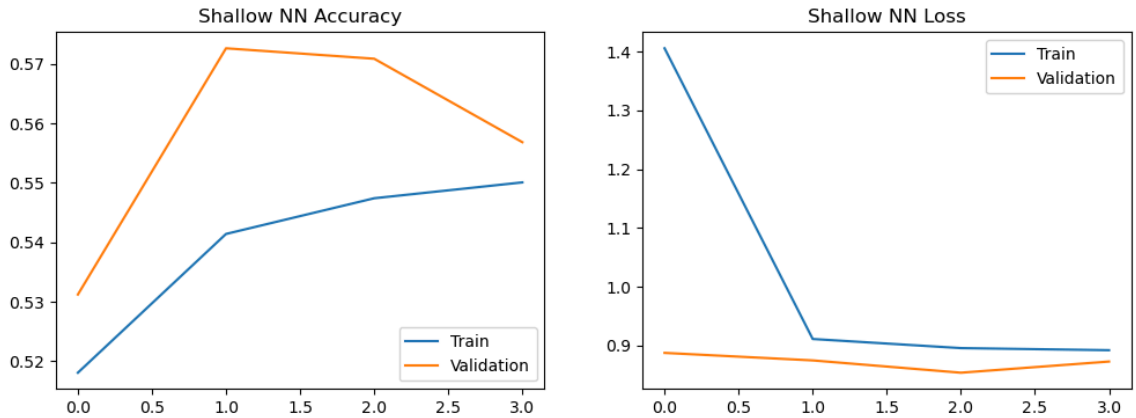
In [14]:
```
history_deep_cnn = model_deep_cnn.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    callbacks=[early_stop]
)
```

```
Epoch 1/10
375/375 [==============================] - 992s 3s/step - loss: 0.4719 - a
ccuracy: 0.8154 - val_loss: 6.6896 - val_accuracy: 0.3333
Epoch 2/10
375/375 [==============================] - 971s 3s/step - loss: 0.2605 - a
ccuracy: 0.9036 - val_loss: 0.3780 - val_accuracy: 0.8418
Epoch 3/10
375/375 [==============================] - 1257s 3s/step - loss: 0.1905 -
accuracy: 0.9302 - val_loss: 1.6266 - val_accuracy: 0.6232
```

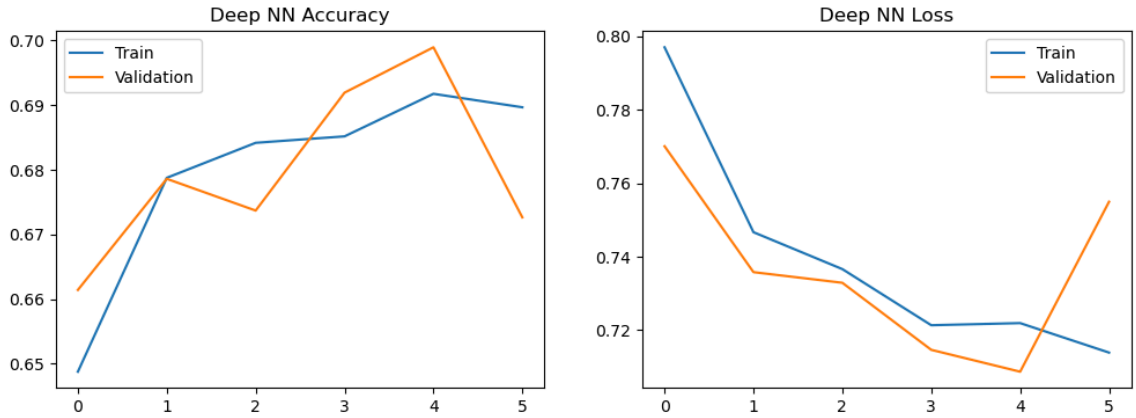
11. Training Curves

In [15]:
```
import matplotlib.pyplot as plt

def plot_training(history, title):

    plt.figure(figsize=(12,4))

    # Accuracy
    plt.subplot(1,2,1)
    plt.plot(history.history['accuracy'])
    plt.plot(history.history['val_accuracy'])
    plt.title(title + " Accuracy")
    plt.legend(["Train","Validation"])

    # Loss
    plt.subplot(1,2,2)
    plt.plot(history.history['loss'])
    plt.plot(history.history['val_loss'])
    plt.title(title + " Loss")
    plt.legend(["Train","Validation"])

    plt.show()
```

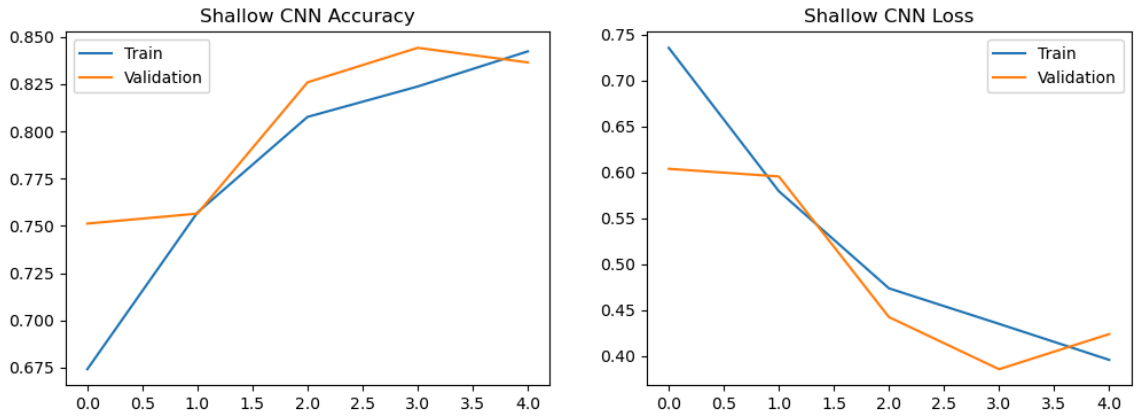
In [16]:
```
plot_training(history_shallow_nn,"Shallow NN")
```



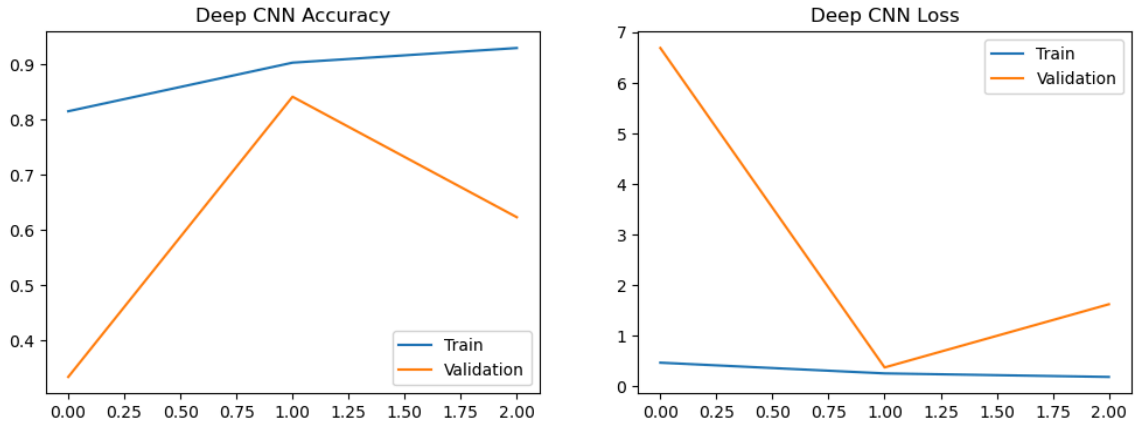
In [17]:
```
plot_training(history_deep_nn,"Deep NN")
```



In [18]:
```
plot_training(history_shallow_cnn,"Shallow CNN")
```



In [19]:
```
plot_training(history_deep_cnn,"Deep CNN")
```



12. Model Evaluation

In [20]:
```
test_loss, test_acc = model_shallow_nn.evaluate(test_generator)

print("Test Accuracy model_shallow_nn:", test_acc)
```

```
5/5 [==============================] - 5s 721ms/step - loss: 0.7275 - accu
racy: 0.6933
Test Accuracy model_shallow_nn: 0.6933333277702332
```

In [21]:
```
test_loss, test_acc = model_deep_nn.evaluate(test_generator)

print("Test Accuracy model_deep_nn:", test_acc)
```

```
5/5 [==============================] - 6s 294ms/step - loss: 0.6544 - accu
racy: 0.6867
Test Accuracy model_deep_nn: 0.6866666674613953
```

In [22]:
```
test_loss, test_acc = model_shallow_cnn.evaluate(test_generator)

print("Test Accuracy model_shallow_cnn:", test_acc)
```

```
5/5 [==============================] - 2s 405ms/step - loss: 0.4653 - accu
racy: 0.8267
Test Accuracy model_shallow_cnn: 0.8266666531562805
```

In [23]:
```
test_loss, test_acc = model_deep_cnn.evaluate(test_generator)

print("Test Accuracy model_deep_cnn:", test_acc)
```

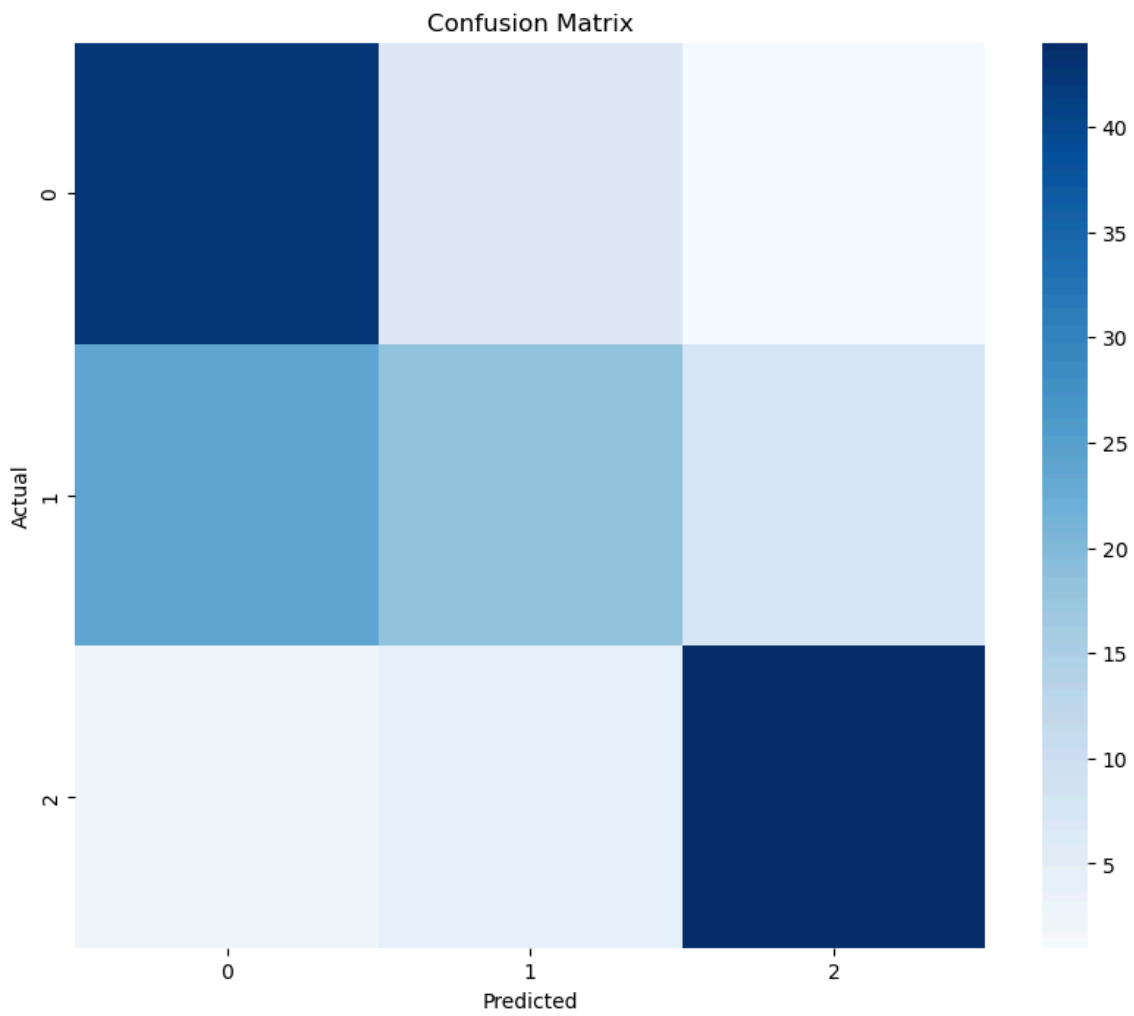
```
5/5 [==============================] - 4s 809ms/step - loss: 0.2057 - accu
racy: 0.9200
Test Accuracy model_deep_cnn: 0.9200000166893005
```

13. Confusion Matrix and Classification Report

In [24]:
```
predictions = model_shallow_nn.predict(test_generator)

y_pred = np.argmax(predictions, axis=1)
y_true = test_generator.classes
```

```
5/5 [==============================] - 3s 352ms/step
```

In [25]:
```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



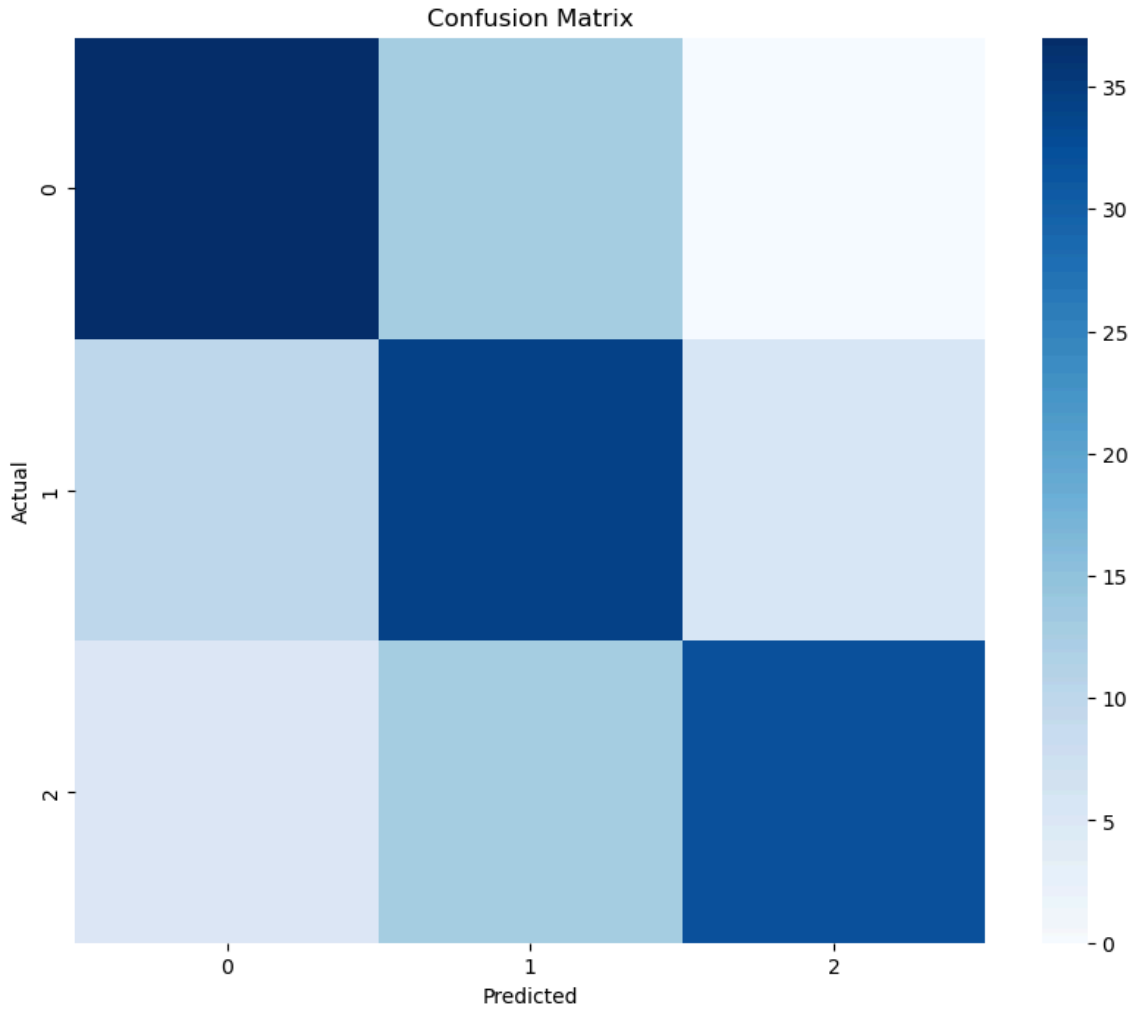
In [26]:
```
print(classification_report(
    y_true,
    y_pred,
    target_names=classnames
))
```

```
              precision    recall  f1-score   support

brain_glioma       0.62      0.84      0.71        50
 brain_menin       0.62      0.36      0.46        50
 brain_tumor       0.83      0.88      0.85        50

    accuracy                           0.69       150
   macro avg       0.69      0.69      0.67       150
weighted avg       0.69      0.69      0.67       150
```

In [27]:
```
predictions = model_deep_nn.predict(test_generator)

y_pred = np.argmax(predictions, axis=1)
y_true = test_generator.classes
```

```
5/5 [==============================] - 2s 356ms/step
```

In [28]:
```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



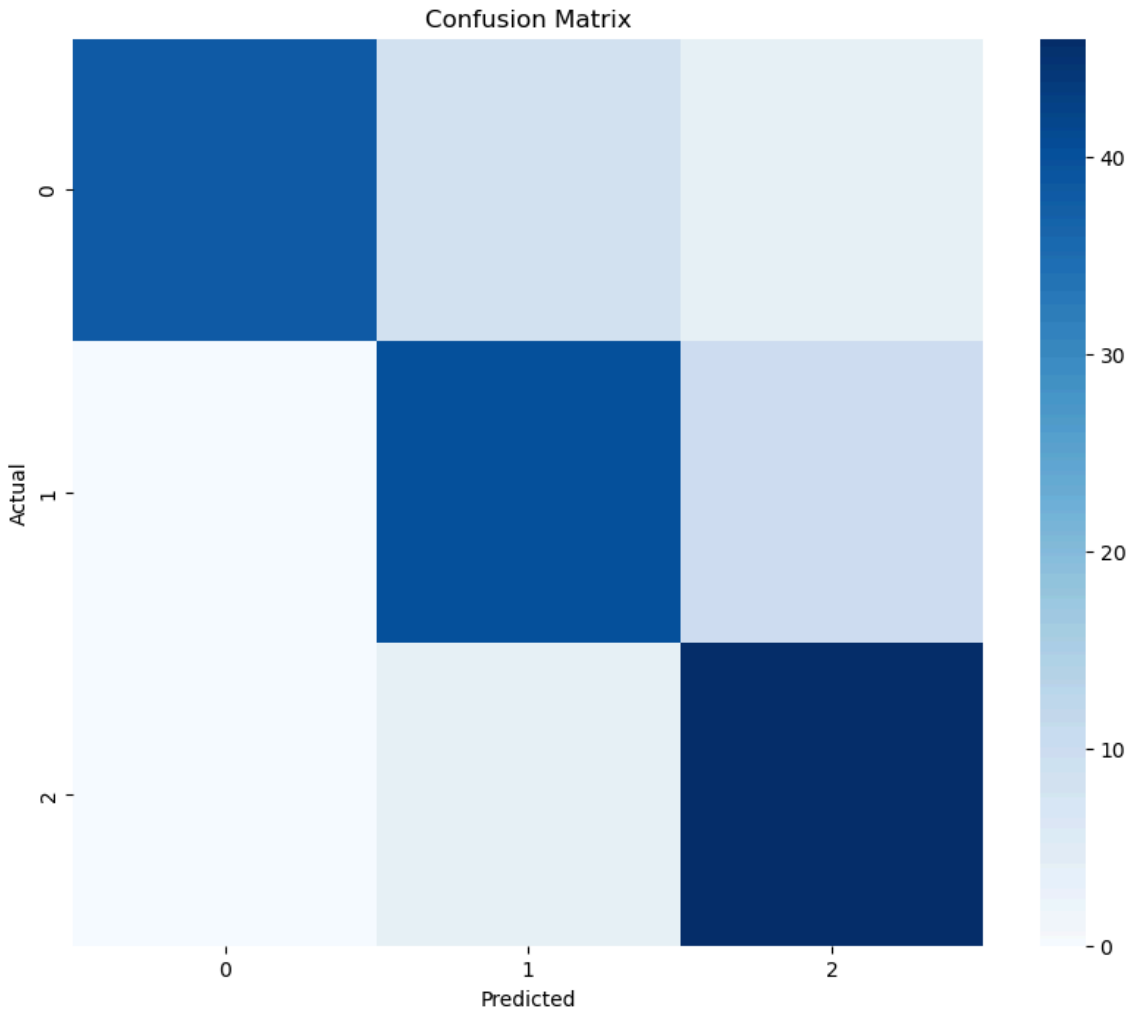
In [29]:
```
print(classification_report(
    y_true,
    y_pred,
    target_names=classnames
))
```

```
              precision    recall  f1-score   support

brain_glioma       0.71      0.74      0.73        50
 brain_menin       0.57      0.68      0.62        50
 brain_tumor       0.84      0.64      0.73        50

    accuracy                           0.69       150
   macro avg       0.71      0.69      0.69       150
weighted avg       0.71      0.69      0.69       150
```

In [30]:
```
predictions = model_shallow_cnn.predict(test_generator)

y_pred = np.argmax(predictions, axis=1)
y_true = test_generator.classes
```

```
5/5 [==============================] - 2s 391ms/step
```

In [31]:
```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



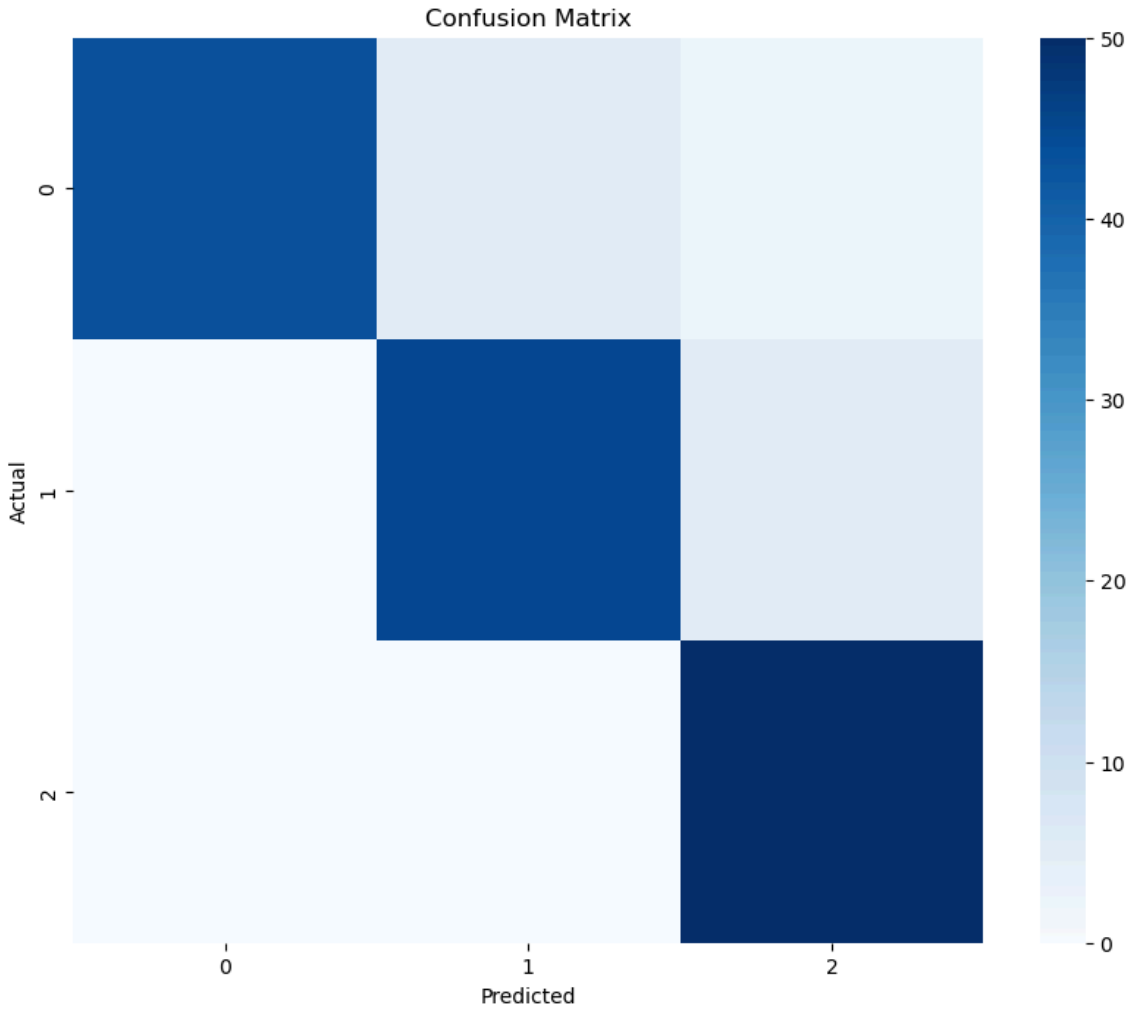
In [32]:
```
print(classification_report(
    y_true,
    y_pred,
    target_names=classnames
))
```

```
              precision    recall  f1-score   support

brain_glioma       1.00      0.76      0.86        50
 brain_menin       0.77      0.80      0.78        50
 brain_tumor       0.77      0.92      0.84        50

    accuracy                           0.83       150
   macro avg       0.85      0.83      0.83       150
weighted avg       0.85      0.83      0.83       150
```

In [33]:
```
predictions = model_deep_cnn.predict(test_generator)

y_pred = np.argmax(predictions, axis=1)
y_true = test_generator.classes
```

```
5/5 [==============================] - 5s 802ms/step
```

In [34]:
```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



In [35]:
```
print(classification_report(
    y_true,
    y_pred,
    target_names=classnames
))
```

```
              precision    recall  f1-score   support

brain_glioma       1.00      0.86      0.92        50
 brain_menin       0.90      0.90      0.90        50
 brain_tumor       0.88      1.00      0.93        50

    accuracy                           0.92       150
   macro avg       0.93      0.92      0.92       150
weighted avg       0.93      0.92      0.92       150
```

14. Save Best Model

In [39]:
```
model_deep_cnn.save("brain_cancer_pred_model.h5")
```

CNN models outperform shallow neural networks because they preserve spatial structure in images. Convolution layers learn local features such as edges, textures, and disease spots directly from the image. Pooling layers introduce translation invariance, meaning the model can recognize patterns regardless of their position in the image. In contrast, shallow fully connected networks flatten the image into a vector, which removes spatial relationships between pixels and makes it harder to detect visual patterns. As a result, CNN architectures achieve significantly higher accuracy in image classification tasks such as brain cancer detection/classification.